A PRACTICAL GUIDE

CareCall

How an AI voice agent for elderly care is built: the stack, the code, and the tradeoffs behind each choice

Telephony, speech, LangGraph, PostgreSQL, tracing and evaluation, with the alternatives you could use on your own next application

Version 1
18 October 2026

Contents

Preface 2

PART I. THE SYSTEM 4

1. What CareCall Is, and the Whole System on One Page 5
2. The Life of One Call, End to End 8

PART II. THE STACK 11

3. The Stack, Concept by Concept 12
4. LangGraph: A Quick Brief and How It Works Here 16
5. Data, Safety and Reliability Concepts 20

PART III. QUALITY 24

6. Observability and Evals 25

PART IV. THE CODE 29

7. The File-by-File Guide 30
8. How the Files Cooperate at Runtime 38

PART V. CHOICES AND ALTERNATIVES 41

9. Alternative Stacks, and the Tradeoffs 42

PART VI. OPERATING IT 53

10. Running It: Commands, Settings, Tests, and What Is Missing 54

Appendix A. Glossary 57

BEFORE YOU BEGIN

Preface

This book explains one working system: CareCall, an AI agent that phones elderly parents on a schedule, asks about meals and medicines, delivers reminders, and reports back to their adult children. It was built as a learning project, so the aim is not only to describe what the code does but to explain *why each piece is there* and what the alternatives would have cost.

Who it is for

An engineer who can read Python and wants to understand how a real-time, model-driven product fits together: how audio travels, how a conversation is kept safe and predictable, how a database acts as both queue and audit log, and how you test something whose output varies. No prior knowledge of telephony, LangGraph or asyncio is assumed; each is introduced when first needed.

How the book is organised

- **Part I, The System.** What the product does and the life of a single call from schedule to follow-up. Read this first.
- **Part II, The Stack.** Each technology in turn (HTTP, FastAPI, asyncio, WebSockets, Twilio, speech, language models, LangGraph, Postgres), with the reasoning for using it and the main tradeoff.
- **Part III, Quality.** Tracing with Langfuse and the evaluation methods: scripted scenarios, simulated parents, judges and regression tests.
- **Part IV, The Code.** A file-by-file guide, then four walk-throughs that show which file hands work to which at runtime.
- **Part V, Choices and Alternatives.** The other stacks you could have used, what each would have changed, and a checklist you can reuse on your own next application.
- **Part VI, Operating It.** Commands, settings, an honest list of what is still missing, and a glossary as an appendix.

One idea to keep in mind

Guarantees live in code; judgment lives in the model. A language model is excellent at understanding messy human speech and poor at being trusted. So the model here only *proposes* structured answers with a quote of what the parent said, and ordinary code validates, decides and writes to the database. Almost every design decision in the following chapters is an application of that sentence.

Status of the system

The book describes the system as of 18 October 2026. The test suite (209 tests on PostgreSQL) passes, the conversation agent, scheduler, outcome engine, post-call agent and live scheduling tools have been exercised with simulated parents, and a live test of in-call scheduling passed all its checks. A real outbound Twilio call has not yet been placed, and several production concerns (migrations, onboarding screens, secrets management) are still open; Chapter 10 lists them.

PART I

The System

What CareCall does, and the life of a single call from schedule to follow-up.

CHAPTER 1

What CareCall Is, and the Whole System on One Page

CareCall phones an elderly parent on a schedule, asks about meals and medicines, delivers one-off reminders, and tells the adult child (the caregiver) what happened. Everything else in the code exists to make that one loop safe, cheap, testable and observable.

The product has five jobs:

- **Plan.** From stored onboarding data (meal times, medicines, reminders, timezone) decide which calls should happen today and write them into the database as scheduled calls.
- **Dial.** At the right minute, one worker claims a due call, checks the parent is not already on a call and that it is not night, and asks Twilio to ring the parent.
- **Talk.** When the parent answers, Twilio streams their voice to our server over a WebSocket. Speech-to-text turns it into words, a LangGraph agent decides what to say, text-to-speech turns the answer into audio, and we stream it back.
- **Decide what happens next.** When the call ends, an outcome engine (rules plus a stronger model) classifies it: completed, cut off, hung up on purpose, never connected, callback requested, emergency, opted out. It schedules the follow-up call and notifies the family when needed.
- **Observe.** Every step is stored as rows and events in Postgres and as traces in Langfuse, and a large test suite plus LLM-played parents check the behaviour.

The design rule behind everything

Guarantees live in code; judgment lives in the model. A model is good at understanding messy speech ("haan, ate dal chawal around one") and bad at being trusted. So the model only *proposes* structured JSON with a quote of what the parent said. Code then validates the quote, applies safety rules, writes to the database through small tools, and chooses the next step. The same rule shapes the post-call agent and the live scheduling tools.

The system in six layers

Layer	What it does	Main technology	Where in the code
Telephony	Rings the phone, carries audio both ways	Twilio Voice + Media Streams (WebSocket)	`telephony/`
Speech	Speech to text, text to speech	Gnani Prisma STT (WebSocket), Gnani Timbre TTS (REST)	`telephony/stt.py`, `tts.py`, `gnani_speech.py`
Conversation	Per-call state machine that decides what to say and which tool to run	LangGraph, Pydantic, an OpenAI-compatible LLM	`agent/`
Scheduling	Plans calls, dispatches them, classifies outcomes, retries	asyncio, SQLAlchemy, Postgres	`calls/`
Storage	Parents, routines, calls, transcripts, logs, events, notifications	PostgreSQL 17 (SQLite for tests)	`models.py`, `db.py`
Quality	Tracing, personas, judges, regression tests	Langfuse, pytest, LLM personas	`tracing.py`, `evals/`, `tests/`

The server is one Python process (FastAPI under uvicorn) that exposes HTTP endpoints for Twilio webhooks and admin views, a WebSocket endpoint for live audio, and optionally runs the scheduler loop in the background.

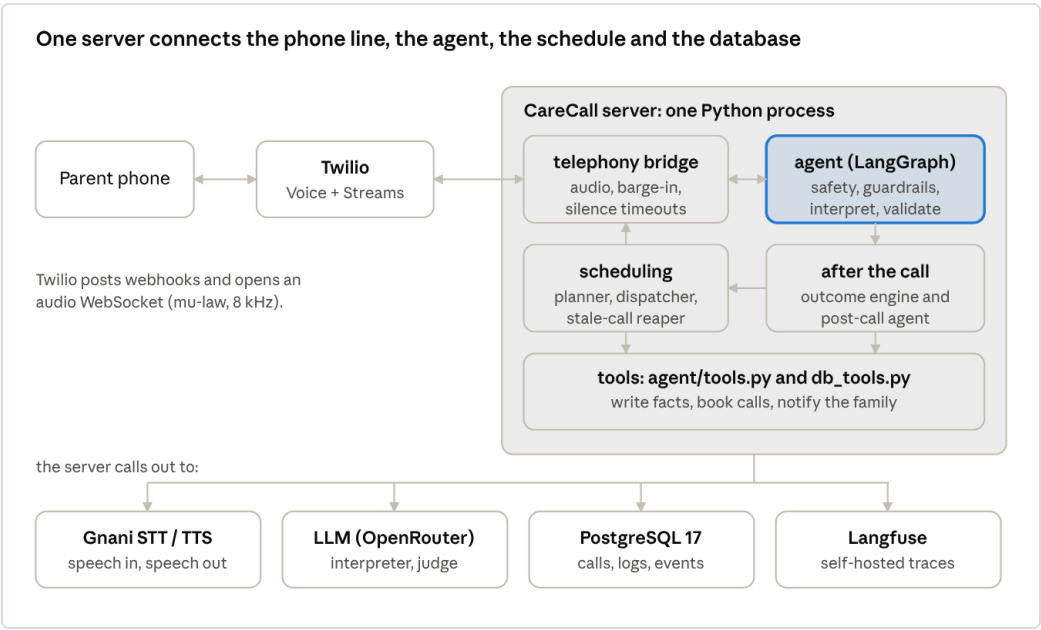


Figure 1. *Architecture at a glance. Twilio and the parent phone talk to the telephony bridge. The bridge feeds the LangGraph agent. Scheduling dials calls (up to the bridge); the agent hands each finished call to the after-call engine (down), which books the next call back into scheduling (left). Everything writes through the tools into PostgreSQL.*

CHAPTER 2

The Life of One Call, End to End

Read this chapter first. Every later chapter explains one piece of it.

Before the phone rings

1. **Onboarding data exists.** A parent row (name, phone, timezone), a caregiver row, a meal profile (usual meal times), medicines (before or after food), and routines such as `lunch_checkin` and `metformin_reminder`. Today this is created by `seed.py`; there is no onboarding screen yet.
2. **The planner turns data into a calendar.** `calls/planner.py` runs hourly (and on `carecall plan-day`). For each parent it works out, in the parent's own timezone, when each routine should fire: a meal check-in is meal time plus 30 minutes, a before-food medicine is meal time minus 15, an after-food medicine is meal time plus its offset, a one-off reminder is its due time. Each becomes a `call_attempts` row with status `scheduled`.
3. **Every planned call gets a key.** The occurrence key is `parent:routine:local-date:slot`, and the first attempt is `<occurrence>:a1`. A unique constraint on that key means running the planner twice creates each call once.

Dialing

4. **The dispatcher claims due calls.** `calls/scheduler.py` selects due rows with `SELECT ... FOR UPDATE SKIP LOCKED`, so several worker processes can run at once and never grab the same call.
5. **Guards run before dialing.** Calling switched off for the parent: cancel. Parent already on a call: postpone 5 minutes. Quiet hours (21:30 to 07:00 local): move to 07:00. Otherwise the row moves to `dialing` and is **committed before** Twilio is asked, so a crash cannot ring the phone twice.
6. **Twilio places the call** (`telephony/twilio_api.py`). When the parent answers, Twilio asks our `/twilio/voice` webhook what to do and we reply with TwiML that

says "open a Media Stream to `wss://.../twilio/stream`".

During the call

7. **A WebSocket opens.** `telephony/bridge.py` accepts it and creates a `CallSession` (`agent/session.py`). The session builds a `CallContext` from the database, opens the LangGraph agent, and the first turn greets the parent and asks the first question.
8. **Text becomes audio.** The reply goes to Gnani TTS, comes back as 16-bit PCM, is resampled to 8 kHz, converted to mu-law, cut into 20 ms frames, base64-encoded and sent down the WebSocket.
9. **Audio becomes text.** The parent's mu-law frames are decoded to PCM and sent to Gnani STT in exact 1024-byte chunks at real-time pace. STT sends back partial and final transcripts.
10. **Each final transcript is one turn.** The graph runs: safety check (emergency words), guardrail (off-topic, abuse, scams), interpret (the model returns JSON with a quote), validate (code checks the quote, hedges, plausibility), act (tools write to the database), advance (next question or goodbye).
11. **Tools run live.** Logging a meal or medicine, recording a meal time, raising an alert, booking a call-back (`book_callback`) or confirming and setting a reminder (`add_reminder`). While a tool runs the agent says a filler such as "Umm... hold on a moment" so the line is never silent.
12. **Barge-in.** If the parent talks while we are speaking, the bridge sends Twilio a `clear` message so our audio stops at once.

After hang-up

13. **The line closes.** The bridge notes whether the parent was mid-speech when the line dropped. `CallSession.finish()` writes the `call.finished` event (steps done, end reason, speech at hang-up) and moves the call to `completed` or `abandoned`.
14. **Post-call work.** A summary and anomaly list are written for the caregiver, then the **outcome engine** (`calls/outcome.py`) classifies the call. For the ambiguous cases a stronger model, the post-call agent (`calls/postcall_agent.py`), reads the transcript, may look things up through database tools, and proposes the next step. Code clamps that proposal with hard rules: the attempt cap of 2, quiet hours, no emergency or opt-out override, and "ring back immediately if the parent was cut off mid-sentence".

15. **The follow-up is scheduled** with a key that matches the original: `<occurrence>:a2` for a retry, `<occurrence>:cb<n>` for a call-back. A call-back booked live is kept, never duplicated.
16. **The family is told** through the notifier when something needs them (we gave up, a worry was voiced, the parent asked us to stop calling).
17. **Tomorrow is planned.** When an occurrence is finished, whether done or given up, the planner adds the same routine for the next day.

The loop across calls

A call is never an isolated event. Cut-off ring-backs, no-answer retries, requested call-backs and next-day scheduling all feed back into the same `call_attempts` table, which the dispatcher polls. That table is the queue, the audit log and the state machine at once.

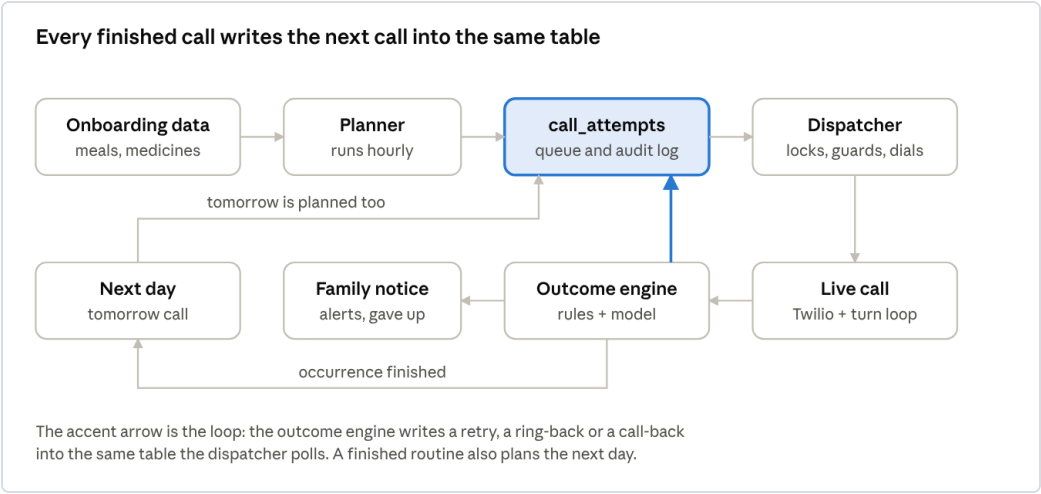


Figure 2. *The call lifecycle loop. The table at the centre is both the queue the dispatcher reads and the record of what happened. A call ends, the outcome engine decides, and its follow-up goes straight back into that table.*

PART II

The Stack

Each technology in turn: what it is, why it is here, and its main tradeoff.

CHAPTER 3

The Stack, Concept by Concept

Each part below says what the technology is, why it is used in CareCall, and what the tradeoff was.

HTTP and REST

HTTP is request and response: a client sends a request, the server sends one response, and the connection is done. **REST** is a style of HTTP API where URLs name things (`/calls/12`) and methods say what to do (`GET` reads, `POST` creates). CareCall uses plain HTTP for everything that is not live audio: Twilio's webhooks (`POST /twilio/voice` asks "what do I do when she answers?", `POST /twilio/status` reports ringing, answered, completed, busy, no-answer), the Gnani post-call webhook, `POST /calls` to start a call by hand, `GET /calls/{id}` to read one, and the `/admin/...` views.

The tradeoff: HTTP is simple, cacheable and easy to test, but it cannot carry a continuous two-way stream with low delay. That is what WebSockets are for.

FastAPI, ASGI and uvicorn

FastAPI is the web framework: you write a Python function, decorate it with `@app.post("/calls")` , and it parses the request, validates it against a Pydantic model and serialises the response. It runs on **ASGI**, the async interface between a Python web app and its server. The server here is **uvicorn**, which accepts network connections and hands each request or WebSocket to the app.

Why this choice: ASGI supports WebSockets and `async` natively, which a phone-call server needs. The older WSGI standard (Flask, Django) is request-per-thread and has no first-class WebSocket story.

One caveat from the code: database calls are synchronous, so inside async handlers the longer ones are pushed to a thread with `asyncio.to_thread(...)` .

asyncio: many calls on one thread

A phone call is mostly waiting: waiting for the parent to speak, for the STT to answer, for the LLM, for TTS audio. **asyncio** runs many such waiting tasks on one thread with an *event loop*. An `async def` function can `await` something slow; while it waits, the loop runs other calls' code.

The pieces used in this project:

- **Tasks** (`asyncio.create_task`): the bridge runs several at once per call, one pumping STT events, one consuming turns, one watching for silence.
- **Queue and Event**: the STT task puts events into an `asyncio.Queue` ; the consumer task takes them out. An `Event` signals "Twilio finished playing our audio".
- `asyncio.to_thread` : the LangGraph turn and all database work are synchronous and may take a second. Running them in a worker thread keeps the event loop free so other calls' audio keeps flowing.
- `run_coroutine_threadsafe` : a thread (the graph, running a tool) can schedule a coroutine back onto the loop. This is how the filler "Umm, hold on..." is spoken while the tool is still running.

Tradeoff: async scales to many calls cheaply, but one blocking call on the loop freezes every call. That is why blocking work is always sent to a thread.

WebSocket

A **WebSocket** starts as an HTTP request that is *upgraded* into a long-lived, two-way channel. Either side can send a message at any time, with small headers and no new connection per message. Two WebSockets exist in this system:

1. **Twilio to us** (`/twilio/stream`): Twilio sends JSON messages with `start` , `media` (base64 audio, one every 20 ms), `mark` ("I finished playing your audio") and `stop` ; we send `media` (our speech), `mark` and `clear` (stop playing now).
2. **Us to Gnani STT** (`wss://api.vachana.ai/stt/v3/stream`): we send raw PCM chunks and receive transcript events.

HTTP streaming (SSE) versus WebSocket. Server-Sent Events let a server push a one-way stream over normal HTTP. Gnani TTS offers it (first audio chunk in 0.06 s instead of 0.56 s for the whole file), and LLMs offer token streaming. CareCall

measured them but does not yet use them in the live path: today the whole sentence is synthesised and then streamed to the phone. Switching to streaming TTS is the biggest remaining latency win.

Twilio Voice and Media Streams

Twilio is the phone network. Placing a call is a REST call to Twilio (`calls.create`) with our webhook URLs. When the parent answers, Twilio fetches TwiML (XML instructions) from us; ours says `<Connect><Stream url="wss://..."/>`, which makes Twilio open the audio WebSocket. Audio is **mu-law, 8 kHz, mono, 20 ms frames (160 bytes)**, which is how telephone networks encode voice. Webhook requests are authenticated with an `X-Twilio-Signature` header that `telephony/twilio_api.py` verifies.

Audio handling

`telephony/audio.py` does the conversions: mu-law to 16-bit PCM and back, resampling between 8 kHz (phone) and the TTS/STT sample rate with a stateful resampler (so chunk boundaries do not click), and framing into 20 ms pieces. The STT needs *exactly* 1024-byte PCM frames delivered at real-time cadence, so the STT client buffers and paces them. Barge-in works by sending Twilio a `clear` message the moment a non-empty partial transcript arrives while we are speaking.

Speech-to-text and text-to-speech (Gnani)

- **STT**: Gnani Prisma over WebSocket, handling Indian languages and code-mixed speech. It emits `speech_start`, partial and final transcripts with confidence.
- **TTS**: Gnani Timbre over REST (`/api/v1/tts/inference`), returning PCM. Voice speed can be changed (slower after the parent asked us to repeat).
- Both sit behind small interfaces (`STTProvider`, `TTSProvider`) so tests can plug in fakes and a text-only channel needs no audio at all.

The language model

The LLM is reached through an **OpenAI-compatible HTTP client** (`llm/openai_compat.py`) pointed at OpenRouter, so the model is a setting, not a code change. Four roles are configured separately in `llm/factory.py`: the *interpreter*

(cheap, no-thinking Qwen for speed during the call), the *judge/post-call agent* (stronger model, runs off the call path), the *persona* (plays the parent in tests) and the optional *model-written agent* used only in experiments.

Important details:

- Thinking is disabled on the fast model through `extra_body` so the reply comes in about a second.
- The client retries transient errors and returns JSON in JSON mode.
- IPv6 is forced off in `net.py` because a broken IPv6 route on this machine made requests wait 40 to 60 seconds.

Pydantic

Pydantic defines typed data models with validation: `Interpretation` (what the model must return), `CallContext` (everything the agent knows about this call), `InterpretRequest`, `Settings`. A model that returns `"reason": null` or a bad field is caught at the boundary instead of crashing deep inside the code.

Settings and environment

`config.py` holds all settings as a Pydantic `Settings` class, read from environment variables and the git-ignored `.env` file: API keys, the database URL, model names, retry policy, quiet hours. Nothing secret is in code.

CHAPTER 4

LangGraph: A Quick Brief and How It Works Here

What LangGraph is

LangGraph is a library for building stateful programs as a graph. You define:

- a **state**: a typed dictionary that every step can read and update;
- **nodes**: ordinary Python functions that take the state and return a *partial update* (only the keys they changed);
- **edges**: which node runs next. A **conditional edge** calls a small routing function that looks at the state and picks the next node, so the program can branch;
- a **checkpointer**: storage that saves the state after each step under a `thread_id`, so the next `invoke` on the same thread continues where the last one stopped.

You compile the graph and call `graph.invoke(input, config)`. Each call runs from the start node, follows edges until it reaches `END`, and returns the final state. The value over plain `if/else` code is that the flow is explicit, each node is traced separately, and state survives between calls.

Why a graph for a phone call

A call is a conversation made of many *turns*, with a plan (the list of questions), memory (what has been answered, how many retries) and branches (emergency, off-topic, "call me later"). A graph gives that structure, plus a clean place to keep guarantees in code. The alternative, a model-written agent that decides everything with free text and tool calls, was built as an experiment (`agent/generative.py`) and plateaued at 14 to 20 of 25 test personas, mostly from skipping steps and wording drift. The graph agent is the production path.

The graph in CareCall

One `invoke` happens per parent utterance (plus one at the start to greet). The graph lives in `agent/graph.py` and is built by `build_graph(interpreter, speak_now)`.

Node	What it does	Model involved?
`open`	First invoke only. Builds the question list from the routine, resumes from a previous cut-off call if there is one, writes `call.opened`, and speaks the greeting plus the first question.	No
`safety`	Regex checks for emergencies (chest pain, fall, can't breathe, real calls for help) and mild symptoms, and runs the guardrail detector. Resets per-turn fields.	No
`guardrail`	Handles out-of-scope requests (stories, news, tasks, OTP or money requests, abuse, prompt injection) with canned replies, strike counting (3 strikes ends politely) and an alert to the family when it matters.	No
`escalate`	Emergency path: raises an emergency alert, tells the parent the family is being contacted, ends the call.	No
`interpret`	Asks the interpreter model to turn the utterance into an `Interpretation` JSON (intent, yes/no, items, time, callback fields, requests, a verbatim quote and confidence). A plain yes/no to a read-back uses simple rules instead.	**Yes**
`validate`	Pure code: the quote must appear in what was said, confidence must be high enough, the answer must have the field the question needs, meal times must be plausible, hedges ("I think", "not sure") are downgraded, filler foods are rejected.	No
`act`	The big decision node. Based on the intent and the current step it calls the tools, picks the words to say, may book a call-back or queue a read-back step, and decides advance, retry or end.	No
`advance`	Moves to the next step, appends the closing line, or ends the call.	No

Routing. From the start, `route_start` sends the first invoke to `open` and every later one to `safety`. After `safety`, an emergency goes to `escalate`, a guardrail hit goes to `guardrail`, anything else goes to `interpret`. After `guardrail`, the turn either ends there (the guardrail already produced the whole reply) or continues to `interpret` when the same utterance also held a real answer. After `validate`, an emergency intent from the model goes to `escalate`, otherwise to `act`. `act` always flows into `advance`.

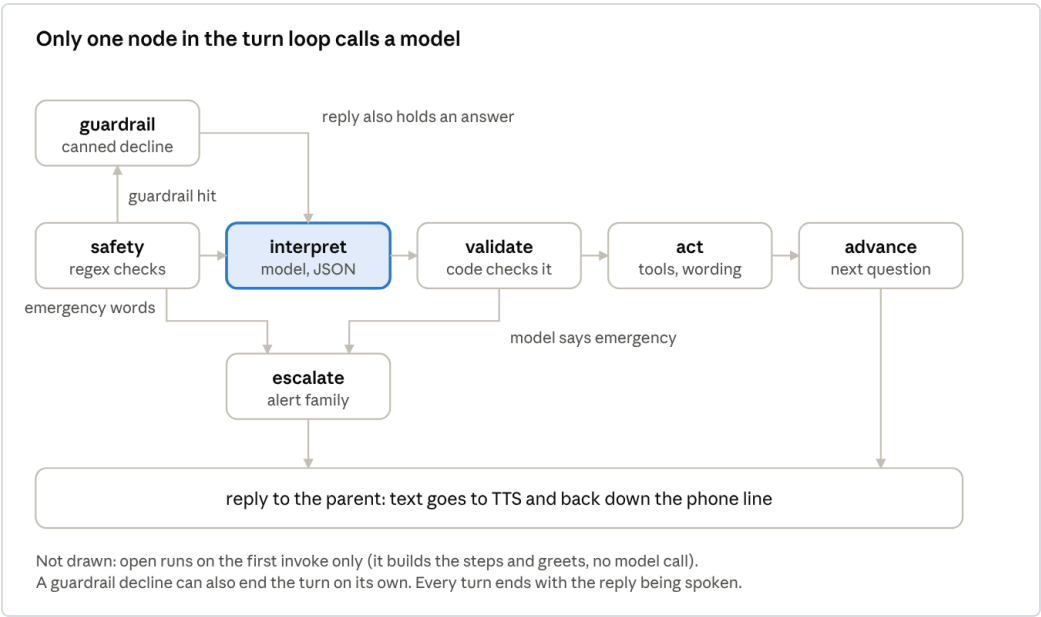


Figure 3. *One turn of the LangGraph agent. The highlighted node is the only place a model is asked. Everything before it is regex, everything after it is code that checks and acts.*

State

The `CallState` holds: the `ctx` (parent, routine, medicines, caregiver, call kind, resume data), the `steps` list and the `idx` pointer, `retries` and `nudges` for the current step, `pending_meal` ("yes I ate" captured, waiting for "what did you eat"), `learned` meal times, `unresolved` steps, `alerted` keys (to avoid spamming the family), `strikes`, guardrail bookkeeping, `early` (lines already spoken live), `filler_n`, the `reply` list and `done` / `end_reason`.

Code, not the model, owns this state. The model never sees or edits it.

Checkpointing and thread ids

The graph is compiled with `MemorySaver`, an in-process checkpointer, and each call uses `thread_id = call-<id>`. That is why `session.turn(utterance)` can pass only the new utterance: LangGraph restores the rest of the state for that thread. The tradeoff is that state lives in the server's memory: if the process crashes mid-call, the call cannot be resumed from the checkpoint. That is acceptable because a dropped

phone call cannot be resumed anyway. Instead, what matters is written to Postgres as it happens (meal logs, alerts, events), and the `call.finished` event records where the conversation stopped so a ring-back can resume from the unanswered question.

Steps, not free conversation

A routine turns into an ordered list of steps (`agent/routines.py`): a meal check-in has "Have you had your lunch today?" and, if yes, "What did you have?"; a medicine reminder has one "Have you taken it?" step with before- or after-food wording; a defined reminder has one acknowledgement step. The wording of every step is code, so reminders are exact and consistent. The model only interprets the reply to the current step.

Prompts by use case

`prompts/base.md` holds the shared interpreter rules (output schema, evidence must be verbatim, how to treat hedges, emergencies, callbacks). Five small files add use-case rules: `meal_discovery` , `meal_checkin` , `medicine_before_food` , `medicine_after_food` , `defined_reminder` . `prompts/__init__.py` joins base plus use case and computes a short hash as the *prompt version*, stored on each call and shown in traces, so any conversation can be tied to the exact prompt text that produced it. At runtime the registry of database tools adds a generated section describing the request tool and its argument schema.

What a node can and cannot do

- Nodes call tools directly (`agent/tools.py` , `agent/db_tools.py`); the model never calls a tool by name during the call.
- Every node is wrapped in `tracing.traced` , so each turn appears in Langfuse as a tree of node spans.
- The graph is deterministic given the interpreter output, which is why most tests replace the interpreter with queued answers and still exercise the whole graph.

CHAPTER 5

Data, Safety and Reliability Concepts

PostgreSQL, SQLAlchemy and psycopg

The database is **PostgreSQL 17**, run in Docker (`docker/app/docker-compose.yml` , port 5433). **SQLAlchemy 2** is the ORM: each table is a Python class in `models.py` , and `session_scope()` in `db.py` gives a transactional session that commits on success and rolls back on error. **psycopg 3** is the driver. The engine uses a connection pool (10 connections plus 20 overflow), `pool_pre_ping` to drop dead connections, and recycling every 30 minutes.

Why Postgres instead of SQLite: SQLite allows one writer at a time and has no row locking, so it cannot safely serve several calls and workers at once. Postgres gives row-level locks, `SELECT ... FOR UPDATE SKIP LOCKED` and real concurrency. SQLite is kept only as a zero-setup option for tests; it returns timezone-naive datetimes, which is why `models.as_utc()` exists.

There are no migrations yet: tables are created with `Base.metadata.create_all` . Alembic is the next step before any production data exists.

The data model

Table	Holds
`parents` , `caregivers` , `parent_caregiver`	Who is called, who is told, and how they are related; parent timezone and the `calling_enabled` switch
`meal_profile`	Usual time per meal, with a source (stated, default, inferred)
`medicines` , `reminders` , `routines`	What to ask about. A routine (`lunch_checkin` , `metformin_reminder` , `reminder_7`) links a type and its config
`call_attempts`	Every call, planned or made: status, channel, Twilio sid, attempt number, idempotency key, occurrence key, kind (scheduled, retry, callback, manual), previous attempt, end reason, outcome
`transcript_turns`	Each line spoken by the agent or the parent, with STT confidence and reply latency

Table	Holds
`meal_logs`, `medicine_logs`, `reminder_acks`	Facts extracted from calls, each with the parent's own words as evidence
`callbacks`	Call-backs promised during a call
`alerts`	Things the family must know, with severity
`call_summaries`, `notifications`	The caregiver-facing summary and the outbox of messages sent
`events`	Append-only audit log: tool calls, interpretations, validation downgrades, outcome decisions
`test_notes`	Notes written by the LLM note writer after evals

The call state machine

A call moves through a fixed set of states, and `calls/state.py` allows only legal moves.

From	To
scheduled	dialing, cancelled (or in_progress directly on the text channel)
dialing	ringing, in_progress, no_answer, failed
ringing	in_progress, no_answer, failed
in_progress	completed, abandoned, failed
completed, abandoned, no_answer, failed, cancelled	final

A **state machine** makes webhooks safe. Twilio may send the same status twice or out of order; an illegal or repeated transition is ignored instead of corrupting the call. Timestamps (`dialed_at` , `started_at` , `ended_at`) are set on the way through.

Idempotency

An operation is **idempotent** if doing it twice has the same effect as doing it once. Three places need it:

1. **Creating a call.** `idempotency_key` has a unique constraint. If two requests or two workers try to create the same key at once, one wins and the others receive the existing row. The insert runs inside a SAVEPOINT (`begin_nested`) so the loser

can recover from the unique-key error without poisoning the whole transaction. This was a real bug found by a 20-worker race test.

2. **Planning.** The key is computed, not random: occurrence key `parent:routine:local-date:slot`, then `<occurrence>:a<n>` for retries and `<occurrence>:cb<n>` for call-backs. Running the planner twice, or the outcome engine twice, cannot create duplicates.
3. **Deciding an outcome.** `outcome.apply` returns the stored decision if `call.outcome` is already set.

Idempotency is not the same as the active-call guard. A new attempt gets a new key, so the key alone cannot stop two different attempts ringing the same parent. That is a separate check, `has_live_call`, used by the dispatcher (postpone 5 minutes) and by `POST /calls` (HTTP 409). The key stops duplicates of the *same* intended call; the guard stops *different* calls overlapping.

Concurrency in the database

Several dispatcher processes may run together. Each runs a query for due calls with `FOR UPDATE SKIP LOCKED`: rows another worker has already locked are skipped, not waited on. The worker flips the row to `dialing` and commits *before* calling Twilio. If the process dies after the commit, the **reaper** (`reap_stale`) notices a call stuck in `dialing` for over 3 minutes (or `in_progress` over 25) and fails it, so the outcome engine retries it. The Postgres tests prove it: 20 racing creates produce one row, and 6 dispatchers dial each of 12 due calls exactly once.

Guardrails and safety in code

Prompts are requests; code is a guarantee. So the rules that must hold are in code:

- **Emergency detection** (`agent/interpret.py`, `detect_emergency`): regex patterns for chest pain, can't breathe, a fall, fainting, bleeding, stroke, and genuine calls for help. It runs *before* any model. A separate check avoids false alarms such as "help me remember".
- **Guardrails** (`agent/guardrails.py`): detectors for off-topic requests (stories, news, recipes), tasks (book, order, cancel), scams (OTP, bank details, money), abuse and prompt injection. Each has a canned reply, a strike count (three ends the call politely), and a rule for when the family is told.

- **Evidence rule**: an answer is accepted only if the model's quote appears in what the parent said (pieces joined by "..." are checked piece by piece).
- **Hedge guard**: "I think" or "not sure" about a medicine is recorded as unconfirmed, never as taken.
- **Plausibility**: a lunch at 3 a.m. is rejected.
- **Length and leak filters** on spoken replies, and an allow-list of tools.

Outcomes and the retry policy

`calls/outcome.py` turns what happened into one of: `completed`, `callback_requested`, `cut_off`, `hung_up_on_purpose`, `never_connected`, `technical_failure`, `emergency`, `off_topic_limit`, `opted_out`, `cancelled`. The policy lives in `config.py`:

- At most **2 attempts per reminder**: the first call plus one more, then the family is told and the system moves on.
- A retry that would land after the **next reminder** is dropped.
- Cut off mid-sentence: ring back **immediately** and resume at the unanswered question.
- No answer: retry once in 10 minutes. Technical failure: once in 5 minutes. Hung up on purpose: once in 30 minutes.
- Parent already on a call: postpone 5 minutes. Quiet hours 21:30 to 07:00 local: move to morning.
- "Stop calling me": switch calling off, cancel pending calls, message the family.

Cut-off detection uses signals recorded at hang-up (was the parent mid-speech, was there pending STT text, did the last line end on "and" or "the", was a question left unanswered) and a goodbye detector. When those signals conflict, a small model decides, and its default is to ring back, the safer mistake.

PART III

Quality

How the agent is traced, tested with simulated parents, and kept honest.

CHAPTER 6

Observability and Evals

Langfuse and OpenTelemetry

Tracing records what a program did, step by step, with timings. **OpenTelemetry (OTel)** is the open standard for it; **Langfuse** is an LLM-focused tracing product built on OTel that adds prompt versions, model-call ("generation") records, scores and datasets. CareCall runs it **self-hosted** from `docker/langfuse/docker-compose.yml` : six containers (web, worker, Postgres, ClickHouse, Redis, MinIO) on port 3000.

Vocabulary used in the project:

- **Trace**: one phone call, from open to post-call work.
- **Span**: a timed step inside it (a turn, a graph node, a tool, the post-call agent).
- **Generation**: one model call, with its prompt, output and model name.
- **Session**: `call-<id>` , grouping everything about one call.
- **User**: `parent-<id>-<name>` , one Langfuse user per real parent, so an admin can open the Users tab and see only that person's calls. Test runs use `eval:<scenario>` so they never mix with real parents.
- **Tags**: channel, routine, suite, persona.
- **Score**: a number attached to a trace (judge scores, pass/fail).

`tracing.py` is a thin wrapper. If no keys are set, or Langfuse is down, every helper becomes a no-op, so tracing can never break a live call. `@tracing.traced(name)` decorates a function as a span (used on every graph node and tool). `call_span` attaches work to a call's trace even when it runs later or in another thread, using a trace id created when the session opens. Deployed Langfuse v4 runs in "events only" mode, so the old `/api/public/traces` endpoints are disabled and `/api/public/v2/observations` is used for queries.

`langfuse_sync.py` (`carecall langfuse-sync`) pushes the prompts and the persona dataset into Langfuse so they can be browsed and diffed in the UI.

A second source of truth is the database itself: the append-only `events` table and the admin endpoints (`/admin/parents` , `/admin/calls` , `/admin/calls/{id}`) show the same story without Langfuse.

Why evals, and the order of authority

An **eval** is a repeatable test of agent behaviour. Because model output varies, CareCall uses layers, each with a clear authority:

1. **Deterministic checks** (`evals/checks.py`) decide pass or fail. Invariants hold for every call (the quote is in the parent's words, replies are short, no medical advice, no duplicate logs); expectations are per scenario (this persona must produce this meal log).
2. **LLM notes** (`evals/notes.py`) explain *why*, citing transcript turns. Code re-checks that every cited turn exists so the note writer cannot approve a log whose quote is missing. A template writer is the offline fallback.
3. **Human review** of the notes and Langfuse traces.

The kinds of evaluation

What	File	What it proves
Scripted scenarios (17 cases in `evals/scenarios/core.yaml`)	`runner.py`	Regression: same inputs, same expected logs, free and fast
Personas (25 cards in `personas/*.yaml` , five per use case)	`personas.py` , `persona_runner.py`	Realistic elderly speakers: hard of hearing, forgetful, chatty, confused, scammed
Guardrail personas (`personas/guardrails.yaml`)	same	A parent asks for a story, a taxi, an OTP; the agent must decline and still do the check-in
LLM parents (`LLMPersona`)	`personas.py`	A model plays the persona, so conversations nobody scripted appear

What	File	What it proves
Model-written agent comparison	`generative_runner.py`	Measures a free-text agent against the same personas (plateaued at 14 to 20 of 25)
Gnani's own agent over real calls	`gnani_runner.py`, `gnani_console.py`, `gnani_judge.py`, `telephony/persona_bridge.py`	Tests Gnani's hosted agent with an LLM parent on a real Twilio line, then judges the transcript against a rubric whose citations are re-checked in code
Voice loopback	`voice_loopback.py`	The audio path end to end (TTS to mu-law to STT) with no Twilio
Latency	`latency.py`	Stage-by-stage timing: STT, LLM, TTS, sequential versus streaming
Model benchmark	`model_bench.py`	Which cheap no-thinking model is fast and still accurate enough
Live scheduling	`live_scheduling.py`	LLM parents book call-backs and reminders; checks the database, the next call, and a human-likeness rubric

Judges and their limits

A judge model grades a transcript against a rubric (warmth, naturalness, stalling, clarity, honesty, pacing) and must quote the lines it relies on. Lessons from calibration: a weak judge makes every score less trustworthy, so the judge model is configured separately and kept stronger than the agent; the judge is never allowed to override a deterministic check; and judge false positives were found and fixed by making it cite evidence that code re-verifies.

The pytest suite

`tests/` holds the regression net, currently 209 tests passing on Postgres. Most replace the interpreter with queued answers or the rule-based interpreter, so they run offline in seconds. Postgres-only tests (`test_concurrency_postgres.py`) run when `TEST_DATABASE_URL` is set. Notable files:

- `test_lifecycle.py`: every call outcome, retries, quiet hours, dispatcher, planner, reaper.
- `test_live_scheduling.py`: live call-back and reminder booking, fillers, honesty about booked times.
- `test_postcall_agent.py`: the post-call model's plumbing and the guards around it.
- `test_db_tools.py`: tool permissions, idempotency, dry-run previews.
- `test_guardrails.py`, `test_persona_regressions.py`, `test_live_audit_regressions.py`: one test for each bug found by a persona or a live run.
- `test_twilio_bridge.py`, `test_audio.py`: the WebSocket bridge and the audio conversions.

PART IV

The Code

Every file, and how the files hand work to one another while a call is running.

CHAPTER 7

The File-by-File Guide

The code is a Python package `carecall` under `src/carecall/`. Think of it as five layers that call downward: **entry points** (`api.py`, `cli.py`) call **conversation and call logic** (`agent/`, `calls/`), which call **adapters** (`telephony/`, `llm/`), which talk to the **database and settings** (`models.py`, `db.py`, `config.py`). Tests and evals sit beside them and drive the same code.

Each table says what the file does and which other files it works with at runtime.

Project root

File or folder	What it is	Works with
`pyproject.toml`	Package definition and dependencies (FastAPI, SQLAlchemy, psycopg, LangGraph, Langfuse, Twilio, httpx, websockets). Also installs the `carecall` command.	`cli.py` is the command entry point
`README.md`	Quick start and the list of commands	-
`.env` and `.env.example`	Secrets and settings (keys, database URL, model names, ngrok URL). `.env` is git-ignored.	`config.py` reads it
`docker/app/docker-compose.yml`	Starts the application Postgres 17 container on 127.0.0.1:5433	`db.py` connects to it
`docker/langfuse/docker-compose.yml`	Starts the self-hosted Langfuse stack (web, worker, Postgres, ClickHouse, Redis, MinIO)	`tracing.py` sends traces to it
`docs/`	`how_it_works.md`, `lifecycle.md` (flow charts and policy numbers), `persona_audit.md`, `gnani_agent_test.md`	Human reading only
`gnani_agent_setup/`	Exported prompts, greetings and closings to paste into a Gnani Agent Builder agent (`carecall gnani-export`)	`gnani_platform.py`, `evals/gnani_console.py`

File or folder	What it is	Works with
`runs/`	Output of eval runs: reports, per-persona notes, results JSON, throwaway databases	`evals/` writes it

Core package files (`src/carecall/`)

File	What it does	Works with
`config.py`	One Pydantic `Settings` object: API keys, model names, database URL and pool size, retry policy (2 attempts, delays), quiet hours, scheduler switch, admin token. `get_settings()` is cached.	Imported by almost every file
`db.py`	Creates the SQLAlchemy engine (Postgres pool, or SQLite for tests), `session_scope()` transaction helper, `init_db()` , `is_postgres()` .	Every file that touches data
`models.py`	All 18 tables as Python classes, plus `utcnow()` and `as_utc()` .	`db.py` creates them; everything reads and writes through them
`net.py`	Builds HTTP and WebSocket clients that force IPv4, working around a broken IPv6 route.	`llm/openai_compat.py` , `telephony/gnani_speech.py` , `gnani_platform.py`
`tracing.py`	Langfuse wrapper: `init_tracing` , `traced` decorator, `call_span` , `score` , `flush` . All no-ops when Langfuse is off.	Used by `agent/` , `calls/` , `llm/`
`api.py`	The FastAPI app: Twilio webhooks, the audio WebSocket, call and admin endpoints, Gnani webhook, persona routes, and the lifespan hook that starts the scheduler when enabled.	`telephony/bridge.py` , `calls/` , `config.py`
`cli.py`	The `carecall` command: `seed` , `chat` , `eval` , `personas` , `call` , `serve` , `plan-day` , `schedule` , `scheduler` , `outcome` , `live-scheduling` , `latency` , `model-bench` , `direct-personas` , `gnani-*` , `twilio-*` , `check-keys` , `langfuse-sync` , and more.	Imports the modules it triggers

File	What it does	Works with
`seed.py`	Creates the demo data: one parent (Asha), one caregiver (Sree), two medicines, a reminder, and their routines.	`cli.py seed`, tests, live evals
`keycheck.py`	`carecall check-keys` tells you what each pasted key can actually do, without printing it.	`config.py`
`langfuse_sync.py`	Uploads prompts and the persona dataset to Langfuse.	`prompts/`, `evals/personas.py`
`gnani_platform.py`	Client for Gnani's Agent Builder platform API (create or update an agent, trigger a call, read logs) and the agent definitions.	`evals/gnani_runner.py`

`agent/` : the conversation brain

File	What it does	Works with
`session.py`	`CallSession`: the object one call lives in. `open()` builds the context and greets; `turn(text)` runs the graph once; `finish()` closes the call, writes `call.finished`, runs post-call work and the outcome engine. Holds the `speak_now` hook for fillers and the Langfuse user and session ids.	Created by `telephony/bridge.py`, `cli.py`, evals; calls `graph.py`, `context.py`, `calls/`
`context.py`	`CallContext`: everything the agent may know about this call (parent, caregiver, routine, medicine, meal times, retry kind, resume data from a previous cut-off call). Built once at dial time and stored as an event.	`session.py`, `graph.py`, `routines.py`
`routines.py`	Turns a routine into ordered steps with exact wording, plus greetings (including "Sorry, we got cut off"), closing lines, spoken times, and the read-back confirm step.	`graph.py`, `context.py`
`graph.py`	The LangGraph agent: nodes, routing, state, validation and the whole `act` decision logic. The largest file.	`interpret.py`, `guardrails.py`, `tools.py`, `db_tools.py`, `routines.py`

File	What it does	Works with
`interpret.py`	The `Interpretation` and `InterpretRequest` models, the `LLMInterpreter` (prompt plus JSON parse plus fallbacks), the `RuleInterpreter` (keyword rules for tests and a baseline), emergency and mild-symptom detection, time parsing.	`llm/`, `prompts/`, `graph.py`
`guardrails.py`	Detectors, canned replies, strike and relay rules for off-topic, scam, abuse, injection and task requests.	`graph.py` (`safety`, `guardrail` nodes)
`tools.py`	The original write tools used by the graph during a call: `log_meal`, `log_medicine`, `set_meal_time`, `ack_reminder`, `schedule_callback`, `relay_message`, `raise_alert`, `record_event`. Each writes a row and an audit event.	`graph.py`, `session.py`
`db_tools.py`	The tool registry with schemas, scopes, argument checks, dry-run and audit: reads (`get_parent_profile`, `get_today_status`, `get_upcoming_calls`) and writes (`schedule_next_call`, `book_callback`, `reschedule_call`, `cancel_call`, `add_reminder`, `update_meal_time`, `notify_caregiver`, `finish_occurrence`, `pause_calls`, `resume_calls`).	`graph.py` (live), `postcall_agent.py`, tests
`generative.py`	An experimental agent where the model writes every reply. Kept for comparison, not for production.	`evals/generative_runner.py`

`calls/`: scheduling, lifecycle and follow-up

File	What it does	Works with
`keys.py`	Builds and parses the idempotency keys (`occurrence_key`, `attempt_key`, `parse_attempt_key`).	`service.py`, `planner.py`, `outcome.py`
`state.py`	The legal state transitions and `transition()`, which also writes `state.changed` events and timestamps.	`api.py`, `scheduler.py`, `session.py`

File	What it does	Works with
`service.py`	`create_call`, `schedule_attempt` (race-safe via SAVEPOINT), `has_live_call`, quiet-hours helpers.	`planner.py`, `scheduler.py`, `outcome.py`, `db_tools.py`
`planner.py`	Turns onboarding data into the day's calls (`plan_day`), schedules the first discovery call (`plan_onboarding`), and the next day's call (`plan_next_occurrence`).	`scheduler.py` loop, `outcome.py`, CLI
`scheduler.py`	`dispatch_once` (claims and dials due calls), `reap_stale` (fails stuck calls), `scheduler_loop` (the background loop).	`api.py` lifespan, `twilio_api.py`
`outcome.py`	Classifies a finished call and acts on it: signals, cut-off evidence, hang-up judge, retries, opt-out, family notices, keeping live bookings, planning tomorrow.	`post_call` pipeline, `postcall_agent.py`, `service.py`
`postcall_agent.py`	The post-call model loop: facts in, optional lookup tools, a JSON decision out, then `enforce()` clamps it with hard rules and falls back to the rules engine on any problem.	`outcome.py`, `db_tools.py`, `llm/`
`post_call.py`	Deterministic after-call pipeline: summary, anomalies, human-review flag, caregiver notification.	`session.py`
`notify.py`	The notifier interface (console in dev, SMS later) and `notify_caregivers`, which writes the outbox row and sends.	`post_call.py`, `outcome.py`, `db_tools.py`

`telephony/`: the phone and audio layer

File	What it does	Works with
`bridge.py`	`StreamBridge`: one per live call. Accepts Twilio's WebSocket, converts audio, feeds STT, runs each final transcript through the session in a worker thread, speaks replies through TTS, handles barge-in, silence timeouts, and hang-up (reports whether the parent was mid-speech). Provides the `speak_now` function for fillers.	`agent/session.py`, `stt.py`, `tts.py`, `audio.py`, `api.py`
`audio.py`	mu-law and PCM conversion, stateful resampling, 20 ms framing.	`bridge.py`, `persona_bridge.py`

File	What it does	Works with
`stt.py`	The `STTProvider` interface, an event type, and a scripted fake STT for tests.	`bridge.py`, tests
`tts.py`	The `TTSProvider` interface and a fake TTS for tests.	`bridge.py`, tests
`gnani_speech.py`	The real Gnani adapters: `GnaniSTT` (WebSocket, exact 1024-byte frames, real-time pacing) and `GnaniTTS` (REST, raw PCM). Both force IPv4.	`bridge.py`, `net.py`
`twilio_api.py`	Places the call, builds the TwiML that opens the media stream, signs the stream token, validates Twilio signatures.	`api.py`, `calls/scheduler.py`
`persona_bridge.py`	The reverse role: an LLM plays an elderly parent on a real call placed by Gnani's own agent, using Gnani STT and TTS. Used for testing Gnani's hosted agent.	`evals/gnani_runner.py`, `api.py` persona routes

`llm/` : model access

File	What it does	Works with
`base.py`	The `LLMClient` protocol, `parse_json` (tolerant JSON extraction) and `complete_json` (calls the model, records a Langfuse generation, parses).	`interpret.py`, `outcome.py`, evals
`openai_compat.py`	HTTP client for any OpenAI-style chat endpoint: retries, JSON mode, `extra_body` (used to disable thinking), `chat()` for multi-turn, temperature variants.	`factory.py`, `net.py`
`factory.py`	Builds the right client for each role: interpreter, agent, judge/post-call, persona.	Everything that needs a model
`gnani.py`	A placeholder adapter for a direct Gnani reasoning-model endpoint. Not implemented because no such public endpoint exists.	-

`prompts/`

File	What it does
`__init__.py`	`system_prompt(usecase)` returns base plus use-case text and a short version hash
`base.md`	Shared interpreter rules and the JSON schema the model must return

File	What it does
`meal_discovery.md`, `meal_checkin.md`, `medicine_before_food.md`, `medicine_after_food.md`, `defined_reminder.md`	Use-case rules added to the base
`gnani/*.md`	Conversation prompts for Gnani's hosted agent (a different job: the model speaks, not just interprets)

`evals/`: testing with simulated parents

File	What it does
`scenarios/core.yaml`	17 scripted scenarios with expected logs
`runner.py`	Runs a scenario in a fresh database and collects checks, notes and files
`personas.py`	Loads persona cards; `ScriptedPersona` and `LLMPersona`
`persona_runner.py`	Runs all personas, builds the audit report and scoreboard
`checks.py`	Deterministic pass or fail rules
`notes.py`	LLM and template note writers with citation re-checks
`generative_runner.py`	Personas against the model-written agent
`gnani_runner.py`, `gnani_console.py`, `gnani_judge.py`	Real phone calls against Gnani's hosted agent, console mode without a platform key, and the transcript judge
`voice_loopback.py`	End-to-end audio test with no Twilio
`latency.py`, `model_bench.py`	Timing and model selection
`live_scheduling.py`	The live test of in-call booking, next-call scheduling and human-likeness

`src/carecall/personas/*.yaml` hold the 25 persona cards plus the guardrail pack: who they are, how they speak, the truth about their day, scripted lines, and what a correct agent must record.

`tests/`

File	Covers
`conftest.py`	Per-test database (SQLite file, or Postgres when `TEST_DATABASE_URL` is set) with seed data
`test_state_and_idempotency.py`	State machine and duplicate creation

File	Covers
`test_flows.py`, `test_llm_paths.py`	Conversation flows with the rule and LLM interpreters
`test_guardrails.py`	Declines, strikes, relays, injection
`test_persona_regressions.py`, `test_live_audit_regressions.py`	One test per bug found by personas and live runs
`test_lifecycle.py`	Outcomes, retries, planner, dispatcher, reaper, resume
`test_live_scheduling.py`	Live booking, read-back, fillers, honesty
`test_postcall_agent.py`	The post-call model loop and its guards
`test_db_tools.py`	Tool schemas, scopes, idempotency, Langfuse user ids
`test_concurrency_postgres.py`	20 racing creates, 6 competing dispatchers (Postgres only)
`test_twilio_bridge.py`, `test_audio.py`	The WebSocket bridge and audio maths
`test_gnani_route.py`	Gnani webhook and persona-on-the-phone routes

CHAPTER 8

How the Files Cooperate at Runtime

Four walk-throughs, each naming the file that does each hop.

Walk-through A: a normal lunch check-in

1. `planner.plan_day` reads `models.Parent`, `MealProfile`, `Routine` and calls `service.schedule_attempt`, which uses `keys.occurrence_key` and inserts a `call_attempts` row inside a SAVEPOINT. Key: `1:lunch_checkin:2026-10-19:s1:a1`.
2. `scheduler.scheduler_loop` (started by `api.py`'s lifespan when `ENABLE_SCHEDULER` is true) wakes every few seconds and calls `dispatch_once`. It locks the due row with `SKIP LOCKED`, checks `service.has_live_call`, moves the status with `state.transition`, commits, then calls `twilio_api.place_call`.
3. Twilio rings the parent, then POSTs to `/twilio/voice` in `api.py`, which returns TwiML from `twilio_api` pointing at `/twilio/stream`. Status callbacks go to `/twilio/status`, which feeds `state.transition`.
4. `api.py` accepts the WebSocket and creates a `bridge.StreamBridge` with a `GnaniSTT` and `GnaniTTS` from `gnani_speech.py`. The bridge builds a `session.CallSession` and runs `session.open()` in a thread. `context.build_context` reads the database; `graph` runs the `open` node; `routines` supplies the greeting and first question.
5. The reply text goes through `GnaniTTS`, `audio.resample_pcm16`, `pcm16_to_mulaw`, `frames`, and out through the WebSocket.
6. The parent says "yes, dal and rice". Frames arrive, `audio.mulaw_to_pcm16` decodes them, `GnaniSTT` returns a final transcript, and `bridge._consume` calls `session.turn` in a thread.
7. `graph` runs `safety`, then `interpret`. `interpret.LLMInterpreter` builds the prompt (`prompts` plus `db_tools.live_tool_prompt`), calls

`llm.base.complete_json`, which uses `openai_compat` and parses JSON into an `Interpretation`.

8. `validate` checks the quote and fields. `act` calls `tools.log_meal`, which writes a `meal_logs` row and an event. `advance` picks the closing line. The session saves `transcript_turns`, and every node and tool shows in Langfuse through `tracing`.
9. The parent hangs up. `bridge._cleanup` records whether they were speaking and calls `session.finish`, which writes `call.finished`, runs `post_call.run_post_call` (summary, anomalies, caregiver notification via `notify`), then `outcome.apply`.
10. `outcome.classify` says `completed`. Because nothing is left to retry, `planner.plan_next_occurrence` schedules tomorrow's lunch call. The loop is closed.

Walk-through B: the line drops mid-sentence

1. While the parent says "yes I had rice and then I...", Twilio closes the stream. `bridge` has been tracking STT `speech_start` events and the energy of incoming frames, so its hang-up report says `parent_speaking: true`.
2. `session.finish` stores that in `call.finished` and the call becomes `abandoned`.
3. `outcome.classify` calls `cut_off_evidence`, sees speech at hang-up and no goodbye, and returns `cut_off` with a follow-up of delay 0. The post-call model is *not* trusted to downgrade this: `postcall_agent.enforce` returns the rules' answer when speech was cut mid-word.
4. `outcome.apply` calls `service.schedule_attempt` with attempt number 2 and the same occurrence key, so the new row is `...:a2`. The cap of 2 attempts means a second cut-off ends in a family notification instead.
5. The dispatcher dials it within seconds. `context.build_context` sees `prev_attempt_id` with outcome `cut_off`, loads the previous `call.finished` event, and `graph.open` resumes from the first unanswered step. `routines.greeting` says "Sorry, we got cut off."

Walk-through C: "Not yet, call me in ten minutes"

1. `interpret` returns intent `callback` or an answer with `callback_minutes: 10`. `validate` accepts it.

2. In `act`, `book()` first calls `filler("callback")`. In a phone call `bridge.speak_now` plays "Umm... hold on a moment, let me note that down." through the event loop while the graph thread continues.
3. `book()` calls `db_tools.call_tool("book_callback", ...)`. The registry checks the arguments and scope, pins `parent_id` from the call context, runs the function, which applies quiet hours, the cap and the next-reminder rule, and creates a `call_attempts` row with key `...:cb2` and `prev_attempt_id` set. It returns the real booked time.
4. `tools.schedule_callback` also records the `callbacks` row. The agent says "Sure, I will call you again in 10 minutes." using the *returned* time, then closes.
5. After hang-up, `outcome.apply` finds the live booking, writes `followup.already_scheduled`, and adds nothing. When that callback call completes, `plan_next_occurrence` schedules tomorrow.

Walk-through D: an emergency

1. The parent says "I fell and my chest hurts". `graph.safety` runs `interpret.detect_emergency` (regex, before any model).
2. The route goes to `escalate`, which calls `tools.raise_alert` with severity `emergency`, says that the family is being contacted, and ends the call with reason `emergency`.
3. `post_call` sends the notification immediately. `outcome.classify` returns `emergency` with no retry, and the post-call model is never asked.

The shared threads through all four

- `models.Event` **rows** are written by `tools`, `state`, `db_tools` and `outcome`, so one call can be reconstructed afterwards from the `events` table.
- **Keys** flow `keys` to `service` to `planner`, `db_tools` and `outcome`, so every layer produces the same key for the same intent.
- `config.Settings` sets the numbers (attempt cap, delays, quiet hours) that `outcome`, `scheduler` and `db_tools` all read.

PART V

Choices and Alternatives

The other stacks you could have used, and the reasoning to reuse on your next application.

CHAPTER 9

Alternative Stacks, and the Tradeoffs

Every technology in this book was chosen against alternatives. This chapter lays those alternatives out, layer by layer, so that on your next application you can make the same kind of decision deliberately instead of copying a stack. The comparisons are qualitative: no prices or benchmark numbers are quoted, because they change quickly and depend on your workload. Product names are examples of a category, not endorsements, and anything you adopt should be checked against its current documentation.

The five forces behind every choice

Almost every decision below came down to the same five forces. Name them before you pick a tool.

1. **Latency.** A phone conversation feels broken if the agent pauses for more than a second or two. This pushed us toward a small, non-thinking model for the live path and away from anything that adds a network hop per turn.
2. **Control versus speed to build.** A managed platform gets you a demo in a day but hides the places where guarantees must be enforced. CareCall needs a guardrail that *cannot* be talked out of, so the control had to live in our code.
3. **Data sensitivity.** The product handles the health and daily routine of elderly people. That favoured self-hosted tracing and a database we own.
4. **Scale and operations.** A handful of parents needs very different infrastructure from a million. We deliberately chose tools with few moving parts.
5. **Team skills and learning.** This was a learning project, so choosing tools that expose the underlying ideas (WebSockets, state machines, SQL locking) had value in itself.

A stack that is right for one weighting of these forces is wrong for another. The tables that follow say which force each choice was optimising.

What we used, in one table

Layer	We used	Main advantage here	Main cost
Phone network	Twilio Voice and Media Streams	Raw audio over a WebSocket, so we keep control of speech and logic; mature documentation	Narrowband 8 kHz audio, per-minute cost, vendor dependence for the number
Speech	Gnani Prisma (STT) and Timbre (TTS)	Built for Indian languages and code-mixed speech such as Hinglish	Smaller ecosystem than the global providers; one more vendor key
Language model	OpenAI-compatible API through OpenRouter; small non-thinking Qwen live, stronger model off-line	One interface and one key; swap a model by changing a string; fast live replies	A middleman in the path; hosted models mean data leaves the machine
Conversation logic	LangGraph with a fixed list of steps	Explicit nodes, routes and state; per-node tracing; resumable state	An extra dependency; for this size a plain state machine would also work
Web and concurrency	FastAPI on ASGI with asyncio	Native WebSockets and async; many calls on one thread	Blocking code must be pushed to threads or it freezes every call
Database	PostgreSQL 17 with SQLAlchemy	Row locks and SKIP LOCKED make it a safe queue; one system for state, queue and audit log	A database queue has a throughput ceiling; no migrations yet
Tracing	Langfuse, self-hosted	Health data stays local; LLM-aware views	Six containers to run and upgrade
Evals	Our own persona harness, judges and pytest	Checks that know the domain (no medical advice, quotes must be verbatim)	We maintain it ourselves

Layer 1: telephony and the voice transport

The decision here is who carries the audio and in what protocol.

Option	Typical advantage	Typical cost	Pick it when
Twilio Media Streams (used)	Mature, well documented, raw audio over WebSocket, strong status webhooks	Higher per-minute cost; 8 kHz mu-law audio limits speech quality	You want control of the speech stack and a dependable path to the phone network

Option	Typical advantage	Typical cost	Pick it when
Other cloud telephony (Telnyx, Vonage, Plivo)	Similar model, sometimes cheaper or with different coverage	Smaller communities; feature gaps differ	Price or number availability decides it
Local Indian providers (Exotel, Ozonetel and similar)	Local numbers, regulatory fit for calling in India	Different APIs; fewer English-language examples	You call at scale inside India and compliance drives the choice
WebRTC stacks (LiveKit, Daily with Pipecat)	Wideband audio, built-in interruption handling, low latency between browsers and apps	You operate more infrastructure; a SIP bridge is needed to reach ordinary phones	The user is in an app or browser, or you want the best audio quality
Managed voice-agent platforms (Vapi, Retell, Gnani Agent Builder)	Fastest route to a working agent, hosting included	The decision loop lives inside the vendor; harder to enforce code-level guarantees or reach your database	A prototype, or a use case where the vendor's tools already cover your needs

Why we chose as we did. Media Streams let us put our own guardrails, database tools and tests around the conversation. A managed platform would have been quicker to start, but Chapter 5 and Chapter 8 depend on code owning the guarantees (emergency detection, the attempt cap, idempotent scheduling). The price is lower audio quality and a vendor dependency on the phone number.

Transport sub-choice. A WebSocket is a long-lived two-way channel and fits real-time audio. Server-Sent Events are simpler for one-way streams such as model tokens. gRPC streaming is efficient between your own services but is not what telephony vendors offer. HTTP request and response is right for webhooks. We used each where it fits and measured SSE for text-to-speech (first audio in 0.06 seconds against 0.56 seconds for the whole file) without yet adopting it live.

Layer 2: speech in and out

Option	Advantage	Cost	Pick it when
Gnani STT and TTS (used)	Indian languages and code-mixed speech	Narrower ecosystem and fewer third-party examples	Your callers speak Indian languages or mix them with English
Global speech APIs (Deepgram, AssemblyAI, Google, Azure)	Mature streaming, broad language lists, good tooling	Accuracy on code-mixed regional speech varies; test with your own audio	English or major languages dominate

Option	Advantage	Cost	Pick it when
Open-source models you host (Whisper variants and others)	Data stays with you; no per-minute fee	You operate GPUs and handle streaming yourself; latency needs work	Privacy or cost at volume outweighs the operational load
Expressive TTS specialists (ElevenLabs, Cartesia)	Very natural voices, low time-to-first-audio	Language coverage and phone-quality tuning differ	Voice quality is the product
Speech-to-speech models (OpenAI Realtime, Gemini Live)	One model hears and speaks; lowest conversational delay, natural interruption	Harder to audit and to enforce deterministic rules; the model, not your code, shapes the wording	Open-ended conversation where exact wording does not matter

The tradeoff that matters. A cascaded pipeline (STT then LLM then TTS) is slower than speech-to-speech, but every stage produces text you can log, check and test. CareCall needs exact reminder wording, verbatim evidence for each logged fact and a dependable emergency path, so it accepts the extra delay and works to reduce it (a small model, streaming TTS as the next step).

Layer 3: language models and how you call them

Where the model runs.

Option	Advantage	Cost
OpenAI-compatible gateway (OpenRouter), used	One API and key for many models; trivially swappable; good for benchmarking	An extra hop; availability and privacy terms of both gateway and provider
Direct provider SDKs	Newest features first; fewer parties	Each vendor needs its own client code
Cloud platforms (Bedrock, Vertex AI, Azure OpenAI)	Enterprise contracts, regional hosting, existing security reviews	More configuration; model catalogue differs
Self-hosted (vLLM, Ollama, llama.cpp)	Full data control; predictable cost at volume	You run, scale and secure inference yourself
A library gateway (LiteLLM and similar)	Unified interface you run yourself	One more component to operate

Which model. A small non-thinking model answers in about a second, which is what a live call needs; a stronger model is used only where latency does not matter (judging, the post-call decision). Choosing one big model for everything is simpler

but makes every turn slower and costlier. Measure with your own prompts before deciding; our benchmark showed the cheaper model understood elderly speech well enough once the code around it validated its output.

How you get structured output. There are four common ways.

Method	Advantage	Cost
Ask for JSON and validate it in code (used)	Works with any model; one round trip; code decides what to trust	Occasional malformed output needs a retry or fallback
Native function or tool calling	Standard, and the model picks the tool	Weaker on small models; extra round trip when tool output goes back to the model
Provider structured outputs or constrained decoding (including libraries such as Instructor and Outlines)	Output always matches the schema	Not available on every model; the shape is guaranteed but not the truth of the content
Free text with parsing	Simplest	Fragile

We used JSON plus validation because the cheap live model is more reliable at filling a fixed schema than at choosing tools, and because the *quote check* (the evidence must appear in what the parent said) catches errors that a schema cannot.

Layer 4: orchestrating the conversation

Option	Advantage	Cost	Pick it when
LangGraph (used)	Explicit nodes, conditional routing and saved state per thread; each node shows up in traces	Another dependency and concepts to learn	The flow branches, state matters and you want checkpointing or human-in-the-loop later
A plain Python state machine	Fewest moving parts; easiest to reason about	You write persistence, tracing hooks and routing yourself	A small, stable flow; honestly sufficient for CareCall's current size
Other agent frameworks (LlamaIndex Workflows, CrewAI, AutoGen, OpenAI Agents SDK, Pydantic AI)	Strong built-in patterns for tool use and multi-agent set-ups	Often assume the model chooses the path, which is the opposite of our guarantee	The model should plan and call tools freely

Option	Advantage	Cost	Pick it when
Durable workflow engines (Temporal, Inngest, Hatchet)	Survive crashes and restarts; excellent for long-running processes	Heavy for a short conversation; more infrastructure	Multi-day processes with retries and timers
Classic dialogue managers (Rasa, Dialogflow CX)	Intent and slot-filling built for scripted dialogue	Less natural with mixed speech; weaker with large language models	A closed menu of intents and no generative step

The honest verdict. LangGraph was chosen partly to learn it and partly for explicit routing and per-node tracing. A plain state machine would also have met today's needs. The graph earns its keep as soon as you add persistence across restarts, branching approvals or parallel work. We also built a model-written agent for comparison; it sounded natural but skipped steps and reached 14 to 20 of 25 test personas, which is why the graph, not free generation, runs production.

Layer 5: web framework and concurrency

Option	Advantage	Cost	Pick it when
FastAPI on asyncio (used)	Native WebSocket and async, Pydantic validation, auto docs	Blocking calls freeze the loop unless moved to threads	Python team, many concurrent waiting tasks
Flask or Django (synchronous)	Huge ecosystems; simple mental model	One thread per connection; WebSockets need add-ons such as Channels	Ordinary request and response apps
Node.js (Fastify, Express)	Excellent event loop; strong real-time ecosystem	Less convenient for ML and data code	The team is TypeScript-first
Go	Cheap concurrency and a single binary	Fewer AI libraries; more code for the model side	Very high connection counts or strict latency
Elixir on the BEAM	Built for millions of lightweight, fault-isolated connections; strong in telecom	Smaller AI ecosystem and talent pool	Telephony at very large scale

Why asyncio. A phone call is mostly waiting: for speech, for the model, for audio. One thread can serve many such calls if nothing blocks it. The cost is discipline: every database call and every graph turn is sent to a worker thread so the audio of other calls never stalls.

Layer 6: data, queues and background work

Option	Advantage	Cost	Pick it when
PostgreSQL as store and queue (used)	One system; the queue is transactional with the state; SKIP LOCKED gives safe multi-worker dispatch	Throughput ceiling far below dedicated brokers; polling	Modest to high thousands of jobs a minute and the need for strong consistency
SQLite	Zero setup	One writer; no row locks	Tests and prototypes
Redis-backed queues (Celery, RQ, Dramatiq)	Mature retry and scheduling patterns; very fast	A second system whose state can drift from the database	Many short background tasks, tolerant of eventual consistency
Cloud queues (SQS) or log systems (Kafka, Redis Streams)	Scale and durability beyond a database	Operational weight; ordering and exactly-once are hard	Very high volume or many consumers
Document stores (MongoDB, DynamoDB)	Flexible schema; scale-out	Weaker relational guarantees; unique-key and transaction patterns differ	Large, loosely structured data with simple access patterns
Durable workflow engines (Temporal)	Retries, timers and state handled for you	A new platform to learn and run	Complex, long-lived flows

Why a database queue. A call's status, its idempotency key and its place in the queue are the same row. Claiming it, marking it dialing and committing is one transaction, which removes a whole class of duplicate-dial bugs. The tradeoff is scale: this design is excellent for thousands of calls an hour and the wrong tool for millions.

ORM and migrations. SQLAlchemy 2 was chosen for portability between Postgres and SQLite tests and for explicit transactions and SAVEPOINTs. Raw SQL with a thin driver is faster to reason about for a few queries; SQLModel and Django's ORM trade flexibility for convenience. Alembic is the planned migration tool and is the main missing piece before production.

Layer 7: tracing and evaluation

Option	Advantage	Cost
Langfuse, self-hosted (used)	Open source, LLM-aware views, data stays on your machines	Six containers to run and upgrade

Option	Advantage	Cost
Hosted LLM observability (LangSmith, Braintrust, Helicone, Weave)	No infrastructure; polished UI and eval tooling	Sensitive transcripts leave your control; per-seat or usage cost
Arize Phoenix	Open source and light to start	Smaller feature set for prompt management
Plain OpenTelemetry with a general backend	One tracing system for everything	You build the LLM-specific views

For evaluation, off-the-shelf frameworks (promptfoo, DeepEval, Ragas, Inspect) speed up common patterns such as retrieval scoring or prompt comparison. We wrote our own harness because the important checks here are specific to the domain: a log is only valid if the parent's quote supports it, the agent must never give medical advice, and persona behaviour must match a hidden ground truth. The cost is that we maintain the harness; the benefit is that each failure becomes a precise regression test.

Layer 8: packaging and deployment

CareCall runs with Docker Compose for the databases and a plain process for the server. That is right for development and a first pilot. Alternatives matter once you deploy:

Option	Fits	Watch out for
A virtual machine or Compose on one host	Pilot, small scale	You manage uptime, backups and upgrades
Container platforms (ECS, Cloud Run, Fly, Render)	Managed scaling and deploys	Long-lived WebSockets and timeouts need checking; sticky behaviour for in-memory call state
Kubernetes	Large teams and many services	A lot of operational overhead for one service
Serverless functions	Short stateless webhooks	Poor fit for a live audio WebSocket that stays open

Note one design consequence: the per-call conversation state sits in process memory. That is fine while each call stays on one server, and it means a load balancer must keep a call's WebSocket on the same instance.

Components CareCall did not need, but your next application might

- **Retrieval and vector search.** If the agent must answer from documents (policies, manuals, medical leaflets), add a vector index. Postgres with the pgvector extension keeps one database; dedicated stores (Qdrant, Weaviate, Pinecone, Chroma) scale larger or offer richer filtering. Judge retrieval quality with its own metrics, not only the final answer.
- **Long-term memory.** Summaries of past calls could inform the next one. Start with plain tables and a summary field before adding a memory framework.
- **Caching and rate limiting.** Redis in front of model calls reduces cost for repeated prompts; add it when you can measure a hit rate.
- **Fine-tuning.** Consider it only after prompts, validation and evals plateau; a fine-tuned small model can cut latency and cost on a narrow task.
- **MCP (Model Context Protocol).** A standard way to expose tools to models across applications. We used an in-process registry because everything runs in one codebase; MCP becomes attractive when several applications or assistants should share the same tools. Whatever the transport, the same rules apply: pin the user identity server-side, check arguments, log every call.
- **Content moderation services.** A provider moderation API can supplement, never replace, deterministic guardrails.

Four reference stacks you can adapt

Application	A sensible starting stack	Why
Voice agent (this book)	Telephony vendor with raw audio, cascaded STT, small LLM, TTS, a graph or state machine, Postgres, Langfuse	Control and testability; text at every stage
Chat or RAG assistant	Web framework, a hosted LLM, pgvector or a vector store, streaming responses over SSE, tracing with an LLM tool	Streaming text and good retrieval matter more than telephony
Document processing pipeline	Object storage, a queue, batch LLM calls with structured outputs, a validation step, human review UI, Postgres	Throughput and correctness; no live latency constraint, so use the strongest model
Long-running agent workflow	A durable workflow engine for state and retries, tool calling with strict permissions, audit log, evals on trajectories	Crashes and timers dominate; each step must be resumable and auditable

A decision checklist for your next application

1. **What is the latency budget?** Under two seconds per turn forces small models, streaming and few network hops; minutes or hours lets you use the strongest model and simple batch design.
2. **What must be guaranteed?** List the rules that can never be broken (no advice, no duplicates, always escalate). Put them in code, and test them first.
3. **Who owns the data?** Health, finance and children's data usually push toward self-hosted tracing, regional hosting and minimal prompts.
4. **How much traffic, and how spiky?** Pick the simplest infrastructure that survives ten times today's load, not one hundred times.
5. **Does the model plan, or does your code plan?** Free planning is flexible but unpredictable; a fixed flow with model interpretation is predictable and testable. Mix them by putting autonomy where mistakes are cheap.
6. **Can you observe and replay a failure?** If you cannot reconstruct what the system knew and decided, you cannot fix it. Choose tracing and an append-only event log early.
7. **How will you test non-deterministic behaviour?** Combine deterministic checks, simulated users, a judge with cited evidence, and one regression test per bug found.
8. **What is the cost of being wrong, and who pays?** The answer decides how much confirmation, human review and conservative defaulting the product needs (CareCall's default is to ring back, the safer mistake).

What we would change first if requirements grew

- Adopt **streaming text-to-speech** and token streaming in the live path to cut the pause before the agent speaks.
- Add **Alembic migrations** and a managed Postgres with backups.
- Move call state from process memory to a store that survives a restart, or add sticky routing, before running more than one server for live calls.
- Add an **onboarding** interface and caregiver authentication, and replace the console notifier with SMS or push.
- Consider **WebRTC or a speech-to-speech model** for a future app-based product, accepting less deterministic wording in return for more natural interaction, but only for conversations where that wording is not safety-critical.

None of these change the central idea. Whatever tools you choose, keep the model's job narrow, put the guarantees in code, record everything as events, and test with simulated users before real ones.

PART VI

Operating It

Commands, settings, and an honest list of what is still missing.

CHAPTER 10

Running It: Commands, Settings, Tests, and What Is Missing

First-time setup

1. Activate the virtual environment: `. .venv/Scripts/activate` (Git Bash on Windows).
2. Start the application database: `docker compose -f docker/app/docker-compose.yml up -d`. Use `127.0.0.1`, not `localhost`.
3. Optional but recommended, start Langfuse: `cd docker/langfuse && docker compose up -d`, then `carecall langfuse-sync`.
4. Fill `.env` (copy `.env.example`): `DATABASE_URL`, the Gnani speech key, the LLM base URL, key and model names, Langfuse keys, and later the Twilio account id, token, number and `PUBLIC_BASE_URL` (an ngrok URL Twilio can reach).
5. Create demo data: `carecall seed`.

Commands you will use

Command	What it does
`carecall chat`	Talk to the agent in the terminal (text channel, nothing is dialed)
`carecall eval`	Run the 17 scripted scenarios with checks and LLM notes
`carecall personas [--mode llm]`	Run the 25 personas (scripted, or played by a model) and write an audit report
`carecall direct-personas`	Same personas against the model-written agent
`carecall plan-day [--date D]`	Create the day's calls from stored data (safe to repeat)
`carecall schedule`	Show the calendar of calls

Command	What it does
`carecall scheduler`	Run the dispatcher: plan, reap stale calls, dial due calls
`carecall outcome <call id>`	Show or apply the outcome decision for one call
`carecall live-scheduling`	The live in-call booking test with LLM parents and a report
`carecall serve`	Start the FastAPI server (webhooks, WebSocket, admin endpoints)
`carecall call` / `call-test`	Place a call through Twilio, or a test call
`carecall latency`, `model-bench`	Measure stage timings and compare models
`carecall gnani-setup`, `gnani-export`, `gnani-personas`	Work with Gnani's hosted agent
`carecall twilio-setup`, `twilio-otp`, `voice-selftest`, `trace-check`, `check-keys`	Connectivity checks
`carecall langfuse-sync`	Push prompts and the persona dataset to Langfuse

Run the tests with `python -m pytest -q`. To include the Postgres-only concurrency tests set `TEST_DATABASE_URL=postgresql+psycopg://carecall:carecall@127.0.0.1:5433/carecall` first.

Settings worth knowing

Setting	Default	Meaning
`max_attempts_per_occurrence`	2	Calls per reminder before giving up
`cutoff_retry_delay_s`	0	Ring back at once after a cut-off
`unreachable_delays_min`	10	Retry delay after no answer
`technical_delays_min`	5	Retry delay after a failed call
`hangup_retry_delay_min`	30	Retry delay after a deliberate hang-up
`postpone_live_call_s`	300	Wait when the parent is on another call
`quiet_start` / `quiet_end`	21:30 / 07:00	No calls at night, parent-local
`stale_dial_s` / `stale_call_s`	180 / 1500	When the reaper fails stuck calls

Setting	Default	Meaning
`scheduler_interval_s`	10	Dispatcher tick
`enable_scheduler`	false	Start the scheduler inside the server
`postcall_agent_enabled`	true	A model decides the next step after a call, guarded by rules
`admin_token`	empty	If set, `/admin/*` needs a bearer token

What is still missing

- **Migrations.** Tables come from `create_all`; Alembic is needed before real data.
- **Onboarding.** There is no signup or setup API or screen; data comes from `seed.py`.
- **Real Twilio and Gnani end-to-end.** The Twilio account details are not configured here yet, so no real outbound call has been placed. Gnani's hosted agent has no public direct reasoning endpoint, so `llm/gnani.py` is a stub.
- **Streaming in the live path.** TTS and LLM streaming were measured but are not used; the whole reply is synthesised first. This is the largest latency win left.
- **The phone-line filler.** The filler hook is tested in text sessions; it has not been tried on a real Twilio call.
- **Voice requests beyond reminders.** Rescheduling or cancelling a call by voice is not exposed to the interpreter yet.
- **Adversarial live scenarios** for the new reminder flow (injection inside a reminder request, many requests in a row).
- **Opt-out during the call.** "Stop calling me" is acted on after the call ends; the agent keeps talking until then.
- **Production concerns:** authentication on the admin endpoints beyond a shared token, secrets management, rate limits, backups and a notification channel beyond the console.

APPENDIX A

Glossary

Term	Meaning in this project
ASGI	The async interface between a Python web app and its server; what lets FastAPI do WebSockets
asyncio	Python's single-threaded concurrency for waiting tasks, driven by an event loop
Attempt	One row in `call_attempts`: a planned or made call
Barge-in	The parent speaks over the agent; we send Twilio `clear` so our audio stops
Caregiver	The adult child who is notified
Checkpointer	LangGraph storage that saves state per `thread_id` between invokes
Cut-off	A call that dropped while the parent was mid-sentence or a question was unanswered
Dispatcher	The loop that claims due calls and dials them
Event	An append-only audit row in `events`
Evidence	The verbatim words of the parent that justify a logged fact
Filler	A short human stall such as "Umm... hold on a moment" spoken while a tool runs
Generation	A recorded model call in Langfuse
Guardrail	A rule in code that decides what the agent may say or do, regardless of the model
Idempotency key	A unique string that makes "create this call" safe to repeat
Interpretation	The structured JSON the model returns for one parent reply
LangGraph	The library that runs the per-call graph of nodes with saved state
Media Stream	Twilio's WebSocket carrying live call audio
mu-law	The 8-bit telephone audio encoding at 8 kHz
Occurrence key	`parent:routine:local-date:slot`, the identity of one planned call
Outcome	The classification of a finished call (completed, cut_off, and so on)
Persona	A simulated parent with a profile and a truth about their day
Post-call agent	The stronger model that proposes the next step after a call, clamped by rules
Reaper	The job that fails calls stuck in dialing or in progress

Term	Meaning in this project
SAVEPOINT	A nested transaction that lets one failed insert be undone without losing the rest
SKIP LOCKED	A Postgres option so concurrent workers skip rows already claimed
SSE	Server-Sent Events, a one-way HTTP stream (offered by TTS and LLMs, not yet used live)
STT / TTS	Speech-to-text and text-to-speech
Tool	A small function that reads or writes the database, with a schema and a scope
TwiML	Twilio's XML instructions for what to do with a call
Turn	One parent utterance plus the agent's reply; one `graph.invoke`
WebSocket	A long-lived two-way connection started as an HTTP upgrade